

Phase 5: SaaS Installation Flow & Billing - Implementation Guide

Created: 2024-12-29

Purpose: Roadmap for converting Qryx from a simple Shopify OAuth app to a full-fledged SaaS platform



Critical Discovery

Date: 2024-12-29

Issue: The original Qryx installation flow was designed as a **direct Shopify OAuth app**, which would allow anyone to install without payment or account creation.

Business Requirement: JNX-OS is a **multi-product SaaS platform** where customers must:

1. Register an account
 2. Browse available products (Qryx, future products)
 3. Select a pricing plan
 4. Complete payment via Stripe
 5. THEN proceed with product-specific setup (e.g., Shopify OAuth for Qryx)
-



Flow Comparison

✗ Current (WRONG):

```
Shopify App Store → "Install Qryx"
↓
Direct redirect to Shopify OAuth
↓
Merchant approves permissions
↓
Callback: Create user, install widget
↓
Done (NO PAYMENT, NO ACCOUNT)
```

 **Required (CORRECT):**

Shopify App Store → "Install Qryx"



www.jnxlabs.ai/api/qryx/install?shop=merchant.myshopify.com



Save shop parameter in encrypted session



Redirect to /login (or /signup for new users)



NEW USER: Registration Form

- Email, Password, Name
- Clerk User + Org created



Redirect to /products (or directly to /products/qryx/setup)



PRODUCT SELECTION

- Qryx (AI Sales Assistant)
- Other JNX products (coming soon)



User selects Qryx → Redirect to /products/qryx/setup



QRYX PRICING PLANS

- STARTER - \$29/mo
- 1,000 conversations/mo
 - Basic customization

- PROFESSIONAL - \$99/mo ★ POPULAR
- 10,000 conversations/mo
 - Full customization
 - Advanced analytics

- ENTERPRISE - Custom Pricing
- Unlimited conversations
 - Dedicated support
 - Custom integrations



User clicks "Subscribe" on chosen plan



STRIPE CHECKOUT SESSION

- Payment method collection
- Stripe Subscription created
- Webhook: checkout.session.completed



✓ Payment Success



Subscription stored in DB (billing_customers table)



NOW: Trigger Shopify OAuth Flow



Retrieve saved shop parameter from session



Redirect to Shopify OAuth:
<https://merchant.myshopify.com/admin/oauth/authorize?...>
 ↓
 Merchant approves permissions
 ↓
 Callback: /api/qryx/callback?code=xxx&shop=merchant.myshopify.com
 ↓

INSTALLATION & SETUP

1. Link shop to user's organization
2. Install widget (Script Tag)
3. Create initial Qryx configuration

↓
 Redirect to Dashboard: /app/qryx
 ↓

QRYX DASHBOARD

- ✓ Connected to: merchant.myshopify
- Subscription: Professional Plan
- Usage: 234 / 10,000 conversations
- Widget Configuration
- Analytics & Reports

Required Components

1. Shop Session Management NOT BUILT

File: lib/session/shop-session.ts

Purpose: Preserve the `shop` parameter through multiple redirects:

```
/api/qryx/install?shop=xyz → /login → /products → /checkout → /oauth
```

Implementation Options:

- **Option A:** Encrypted session cookie (iron-session or jose)
- **Option B:** Database state table with expiry

Functions Required:

```
export async function setShopSession(shop: string): Promise<void>;
export async function getShopSession(): Promise<string | null>;
export async function clearShopSession(): Promise<void>;
```

2. Product Selection Page NOT BUILT

File: app/products/qryx/setup/page.tsx

Features:

- Display Qryx pricing tiers (Starter, Professional, Enterprise)
- Plan comparison table

- “Subscribe” button per plan
- Check if user already has a Qryx subscription (redirect to dashboard if so)

UI Requirements:

- JNX Dark design system
 - Responsive pricing cards
 - Clear feature comparison
 - Prominent CTA buttons
-

3. Stripe Integration ❌ NOT BUILT

3a. Stripe Client Setup

File: lib/stripe/client.ts

```
import Stripe from 'stripe';

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2024-12-18',
  typescript: true,
});

export const PRICING_PLANS = {
  starter: {
    name: 'Starter',
    priceId: 'price_xxx', // Stripe Price ID
    amount: 2900, // $29.00
    features: ['1,000 conversations/mo', 'Basic customization'],
  },
  professional: {
    name: 'Professional',
    priceId: 'price_yyy',
    amount: 9900, // $99.00
    features: ['10,000 conversations/mo', 'Full customization', 'Analytics'],
  },
  enterprise: {
    name: 'Enterprise',
    priceId: 'price_zzz',
    amount: null, // Custom pricing
    features: ['Unlimited', 'Dedicated support', 'Custom integrations'],
  },
};
```

3b. Checkout Session Creation

File: app/api/stripe/checkout/route.ts

```

import { stripe, PRICING_PLANS } from '@lib/stripe/client';
import { currentUser } from '@clerk/nextjs/server';

export async function POST(request: Request) {
  const user = await currentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const { planId } = await request.json();
  const plan = PRICING_PLANS[planId as keyof typeof PRICING_PLANS];

  if (!plan) return NextResponse.json({ error: 'Invalid plan' }, { status: 400 });

  const session = await stripe.checkout.sessions.create({
    customer_email: user.emailAddresses[0].emailAddress,
    line_items: [
      {
        price: plan.priceId,
        quantity: 1,
      },
    ],
    mode: 'subscription',
    success_url: `${process.env.NEXT_PUBLIC_APP_URL}/api/qryx/start-oauth?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/products/qryx/setup?canceled=true`,
    metadata: {
      userId: user.id,
      planId,
      productType: 'qryx',
    },
  });

  return NextResponse.json({ url: session.url });
}

```

3c. Webhook Handler

File: app/api/stripe/webhook/route.ts

```

import { stripe } from '@lib/stripe/client';
import { upsertSubscription } from '@lib/db/billing-helpers';

export async function POST(request: Request) {
  const body = await request.text();
  const sig = request.headers.get('stripe-signature')!;

  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(
      body,
      sig,
      process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err) {
    return NextResponse.json({ error: 'Webhook signature verification failed' }, { status: 400 });
  }

  switch (event.type) {
    case 'checkout.session.completed':
      const session = event.data.object as Stripe.Checkout.Session;
      await upsertSubscription({
        userId: session.metadata!.userId,
        stripeCustomerId: session.customer as string,
        stripeSubscriptionId: session.subscription as string,
        planId: session.metadata!.planId,
        status: 'active',
      });
      break;

    case 'customer.subscription.updated':
      const updatedSub = event.data.object as Stripe.Subscription;
      await upsertSubscription({
        stripeSubscriptionId: updatedSub.id,
        status: updatedSub.status,
        currentPeriodEnd: new Date(updatedSub.current_period_end * 1000),
      });
      break;

    case 'customer.subscription.deleted':
      const deletedSub = event.data.object as Stripe.Subscription;
      await upsertSubscription({
        stripeSubscriptionId: deletedSub.id,
        status: 'canceled',
      });
      break;

    case 'invoice.payment_failed':
      const failedInvoice = event.data.object as Stripe.Invoice;
      // TODO: Send email notification, suspend access
      break;
  }

  return NextResponse.json({ received: true });
}

```

3d. Billing Database Helpers

File: lib/db/billing-helpers.ts

```

import { createSupabaseServerClient } from '@lib/supabase/server';

export async function upsertSubscription(data: {
  userId?: string;
  stripeCustomerId?: string;
  stripeSubscriptionId: string;
  planId?: string;
  status: string;
  currentPeriodEnd?: Date;
}) {
  const supabase = createSupabaseServerClient();

  const { data: subscription, error } = await supabase
    .from('billing_customers')
    .upsert({
      user_id: data.userId,
      stripe_customer_id: data.stripeCustomerId,
      stripe_subscription_id: data.stripeSubscriptionId,
      plan_id: data.planId,
      status: data.status,
      current_period_end: data.currentPeriodEnd,
      updated_at: new Date().toISOString(),
    }, { onConflict: 'stripe_subscription_id' })
    .select()
    .single();

  if (error) throw error;
  return subscription;
}

export async function getSubscription(userId: string) {
  const supabase = createSupabaseServerClient();

  const { data, error } = await supabase
    .from('billing_customers')
    .select('*')
    .eq('user_id', userId)
    .eq('status', 'active')
    .single();

  if (error && error.code !== 'PGRST116') throw error;
  return data;
}

```

4. Installation Flow Update CRITICAL

File: app/api/qryx/install/route.ts

Current (WRONG):


```
export async function GET(request: NextRequest) {
  const shop = searchParams.get('shop');
  if (!shop) return NextResponse.json({ error: 'Missing shop' }, { status: 400 });

  // WRONG: Direct redirect to OAuth
  const authUrl = await getAuthorizationUrl(shop, generateNonce());
  return NextResponse.redirect(authUrl);
}
```

Required (CORRECT):

```
import { setShopSession } from '@/lib/session/shop-session';

export async function GET(request: NextRequest) {
  const shop = searchParams.get('shop');
  if (!shop) return NextResponse.json({ error: 'Missing shop' }, { status: 400 });

  // Save shop parameter in session
  await setShopSession(shop);

  // Redirect to login
  const loginUrl = new URL('/login', request.url);
  loginUrl.searchParams.set('redirect', '/products/qryx/setup');
  return NextResponse.redirect(loginUrl);
}
```

5. OAuth Trigger After Payment ❌ NOT BUILT

File: app/api/qryx/start-oauth/route.ts (NEW)

Purpose: Triggered AFTER successful Stripe payment

```
import { getShopSession } from '@lib/session/shop-session';
import { getSubscription } from '@lib/db/billing-helpers';
import { getAuthorizationUrl, generateNonce } from '@lib/shopify/client';
import { currentUser } from '@clerk/nextjs/server';

export async function GET(request: NextRequest) {
  const user = await currentUser();
  if (!user) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  // Verify user has active subscription
  const subscription = await getSubscription(user.id);
  if (!subscription || subscription.status !== 'active') {
    return NextResponse.json({ error: 'No active subscription' }, { status: 403 });
  }

  // Retrieve saved shop parameter
  const shop = await getShopSession();
  if (!shop) {
    return NextResponse.json({ error:
'Shop session expired. Please start installation again.' }, { status: 400 });
  }

  // Generate OAuth URL
  const state = generateNonce();
  const authUrl = await getAuthorizationUrl(shop, state);

  // Redirect to Shopify OAuth
  return NextResponse.redirect(authUrl);
}
```

6. Subscription Management UI ❌ NOT BUILT

File: app/app/billing/page.tsx

Features:

- Current plan display (Starter, Professional, Enterprise)
- Usage statistics (e.g., “2,350 / 10,000 conversations this month”)
- Next billing date
- Billing history (invoices)
- “Upgrade Plan” / “Downgrade Plan” buttons
- “Cancel Subscription” button (with confirmation)
- Invoice download links

UI Components:

- Use `FeatureCard` for plan display
- Use `StatusBadge` for subscription status
- Use `ButtonPrimary` for upgrade actions
- Use `ButtonSecondary` for cancel/downgrade



Implementation Phases

Phase 5A - Core Flow (Priority 1) 🎯

Goal: Enable basic SaaS installation flow

Tasks:

1. ☒ Implement shop session management (`lib/session/shop-session.ts`)
2. ☒ Update installation flow redirect (`app/api/qryx/install/route.ts`)
3. ☒ Create product selection page (`app/products/qryx/setup/page.tsx`)
4. ☒ Integrate Stripe checkout (`lib/stripe/client.ts` , `app/api/stripe/checkout/route.ts`)
5. ☒ Implement OAuth trigger after payment (`app/api/qryx/start-oauth/route.ts`)

Success Criteria:

- User can start installation from Shopify App Store
- User is redirected to JNX login
- User can select a pricing plan
- Payment is processed via Stripe
- OAuth happens AFTER successful payment

Phase 5B - Subscription Management (Priority 2) 🔁

Goal: Enable full billing lifecycle management

Tasks:

1. ☒ Implement Stripe webhook handler (`app/api/stripe/webhook/route.ts`)
2. ☒ Create billing database helpers (`lib/db/billing-helpers.ts`)
3. ☒ Build billing dashboard UI (`app/app/billing/page.tsx`)
4. ☒ Implement usage tracking (conversations, API calls)
5. ☒ Add plan upgrade/downgrade logic
6. ☒ Implement subscription cancellation flow

Success Criteria:

- Webhooks correctly update subscription status
- Dashboard displays current plan and usage
- Users can upgrade/downgrade plans
- Users can cancel subscriptions
- Usage limits are enforced

Phase 5C - Polish & Production Readiness (Priority 3) ✨

Goal: Production-grade UX and error handling

Tasks:

1. ☒ Error handling & recovery flows
2. ☒ Email notifications (payment success/failed, subscription expiring)
3. ☒ Admin subscription management (view all customers, suspend/resume)
4. ☒ Analytics & reporting (MRR, churn rate, popular plans)
5. ☒ Implement trial periods (optional)
6. ☒ Add promo codes/coupons (optional)

Success Criteria:

- Clear error messages for all failure scenarios
 - Users receive email confirmations
 - Admins can manage all subscriptions
 - Analytics provide business insights
-

 Testing Strategy

Test Shop:

- **Domain:** shopbotv3.myshopify.com
- **Purpose:** End-to-end testing

Test Scenarios:**Scenario 1: New User Installation**

1. Click "Install Qryx" in Shopify App Store (simulate via direct URL)
2. Redirected to JNX login → Complete signup
3. Redirected to product selection → Choose Professional plan
4. Complete Stripe checkout (use test card)
5. Redirected to Shopify OAuth → Approve permissions
6. Redirected to Qryx dashboard → Verify connection

Scenario 2: Existing User Installation

1. Already logged into JNX
2. Click "Install Qryx" → Redirected to product selection
3. Complete payment → OAuth → Dashboard

Scenario 3: Payment Failure

1. User selects plan → Stripe checkout fails
2. User sees error message
3. Can retry payment

Scenario 4: Subscription Cancellation

1. User navigates to billing dashboard
2. Clicks "Cancel Subscription"
3. Confirms cancellation
4. Widget remains active until period end
5. Access revoked after period end

Scenario 5: Plan Upgrade

1. User on Starter plan
 2. Navigates to billing → Clicks "Upgrade to Professional"
 3. Stripe processes prorated charge
 4. Plan updated immediately
 5. Usage limits increased
-

Critical Considerations

1. Session Expiry

The `shop` parameter must be preserved for ~15-20 minutes to accommodate:

- User registration (1-2 min)
- Product browsing (2-3 min)
- Payment form completion (3-5 min)
- Stripe 3D Secure verification (1-2 min)
- OAuth approval (1-2 min)

Solution: Set session expiry to 30 minutes

2. Webhook Reliability

Stripe webhooks may be delayed or fail. Implement:

- Idempotent webhook handlers (check if subscription already exists)
- Retry logic for failed webhook processing
- Manual reconciliation endpoint for admins

3. Security

- Never expose Stripe Secret Key in client-side code
- Always verify webhook signatures
- Validate user subscription before granting access
- Rate limit checkout session creation

4. User Experience

- Clear progress indicators during multi-step flow
- Save draft progress (e.g., selected plan) if user exits
- Error recovery: “Something went wrong? Click here to restart”
- Email confirmations at each major step

5. Database Constraints

Ensure foreign keys are correct:

- `billing_customers.user_id` → `users.user_id`
 - `shopify_shops.org_id` → `orgs.org_id`
 - `qryx_chat_sessions.user_id` → `users.user_id`
-

Success Metrics

Phase 5A Complete When:

- [] User can complete full installation flow end-to-end
- [] Payment is processed successfully via Stripe
- [] OAuth happens ONLY after payment
- [] Widget is installed automatically
- [] Dashboard shows “Connected” status

Phase 5B Complete When:

- [] Webhooks update subscription status correctly

- [] Billing dashboard displays plan and usage
- [] Users can upgrade/downgrade plans
- [] Users can cancel subscriptions
- [] Usage limits are enforced (e.g., conversations/month)

Production Ready When:

- [] All test scenarios pass
- [] Error handling is comprehensive
- [] Email notifications are working
- [] Admin tools are functional
- [] Security audit passed
- [] Performance testing complete
- [] Documentation updated



Related Documentation

- **CONVERSATION_STARTER.md** - Full flow diagrams and context
 - **ABACUS_AGENT_ONBOARDING.md** - Agent onboarding with Phase 5 details
 - **ENVIRONMENT_VARIABLES_STATUS.md** - Current env var status
 - **docs/QRYX_PRICING_STRATEGY.md** - Pricing tiers and strategy (if exists)
 - **docs/BACKEND_CONTRACT.md** - Protected files and development rules
-



Getting Started

For the next Abacus AI agent continuing this work:

1. **Read this document completely**
2. **Review** `CONVERSATION_STARTER.md` **for context**
3. **Start with Phase 5A, Task 1:** Shop session management
4. **Test each component incrementally**
5. **Update this document as you progress**

Recommended Approach:

- Implement Phase 5A in order (dependencies exist)
 - Test each component before moving to next
 - Use `shopbotv3.myshopify.com` for testing
 - Document any issues or blockers
-

Document Created By: Abacus AI DeepAgent

Last Updated: 2024-12-29 12:25 UTC

Status: Active - Phase 5A Ready to Start

Next Review: After Phase 5A completion